発展プログラミング (Advanced Programming)

5. デザインパターンの実践

尾崎 知伸 ＜ozaki.tomonobu@nihon-u.ac.jp＞

Visitor パターン

～構造を渡り歩きながら仕事をする～

Visitorパターン

- 想定シーン
  - データ構造の中に沢山の要素が格納されている
  - その各要素に対して何らかの「処理」を行う
  - この「処理」はどこに書くべきか？
    - データ構造を表すクラスに「処理」を書く→ NG
    - ∵「処理」が増えるたびにデータ構造クラスを変更

- Visitorパターン：「データ構造」と「処理」を分離する
  - 訪問者：データ構造の中を巡り歩き「処理」を行う
  - データ構造：訪問者を受け入れる

  - 良い点：新しい「処理」の追加が容易
  - 悪い点：訪問先のデータ構造が変わると作り直し

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

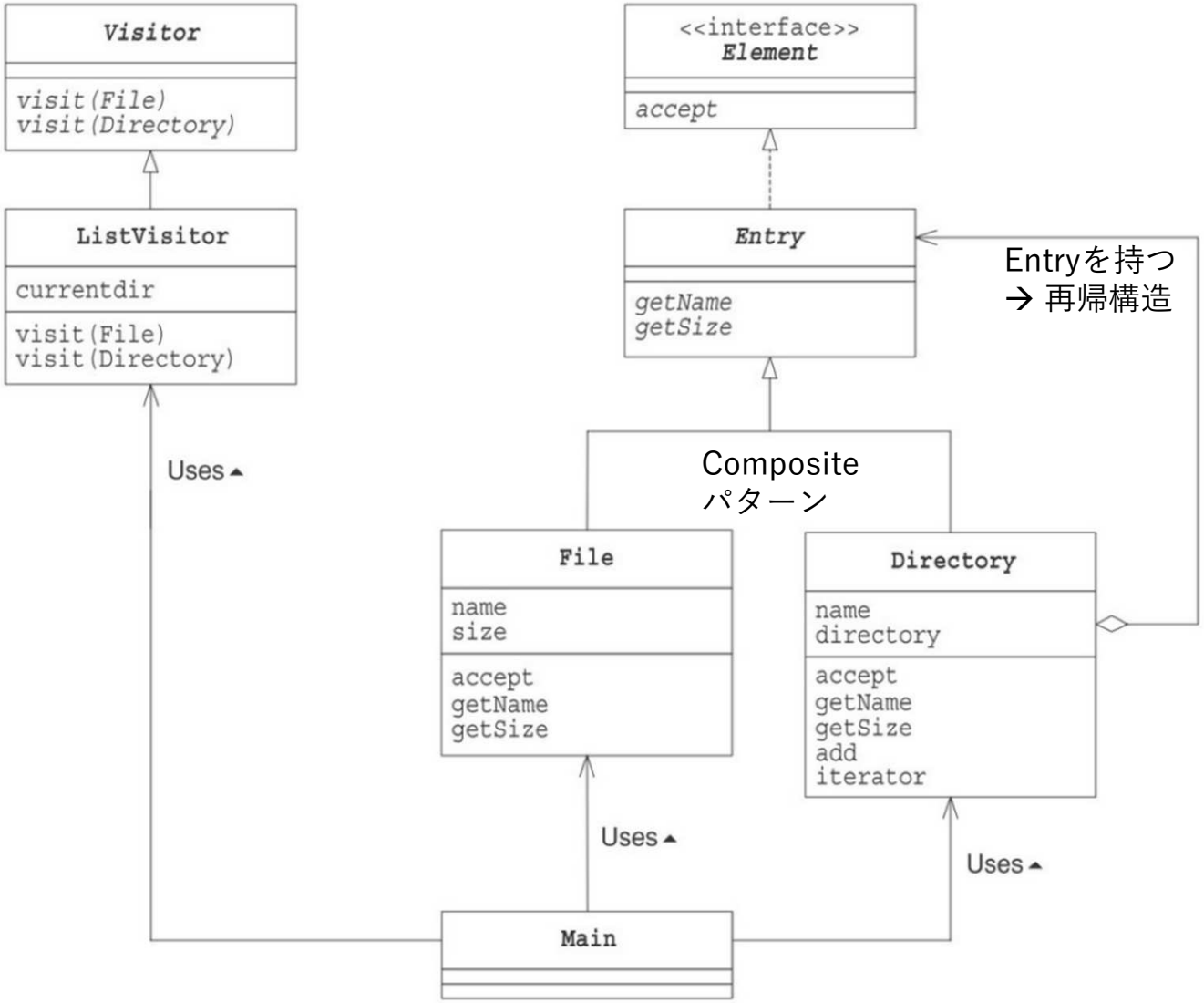
サンプルプログラム

lec05_visitor/*.java

コンパイル方法: javac lec05_visitor/Main.java
実行方法:　　java lec05_visitor/Main

名　前	解　説
Visitor	ファイルやディレクトリを訪れる訪問者を表す抽象クラス
Element	Visitorクラスのインスタンスを受け入れるデータ構造を表すインタフェース
ListVisitor	Visitorクラスのサブクラスで、ファイルやディレクトリの一覧を表示するクラス
Entry	FileとDirectoryのスーパークラスとなる抽象クラス (Acceptorインタフェースを実装)
File	ファイルを表すクラス
Directory	ディレクトリを表すクラス
Main	動作テスト用のクラス

- Visitorがファイル階層を渡り歩く
  - 訪問者：訪問先を分ける
  - 訪問者の訪問先での挙動
    - Visitorの具体クラスで制御
    - 新たな挙動の追加が容易
  - 訪問先（データ構造）
    - Compositeパターン
    - 訪問先の追加
      →訪問者の修正が必要




```
Making root entries...
/root (30000)
/root/bin (30000)
/root/bin/vi (10000)
/root/bin/latex (20000)
/root/tmp (0)
/root/usr (0)
....
```

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Visitorは，対象を訪問
（訪問先クラスを知っている＝訪問先データ構造に依存）

```java
public abstract class Visitor {
    public abstract void visit(File file);
    public abstract void visit(Directory directory);
}
```

訪問先クラスで処理を分ける

抽象クラスの継承
（抽象メソッドの実装）

```java
public class ListVisitor extends Visitor {
    // 現在注目しているディレクトリ名
    private String currentdir = "";

    // ファイルを訪問したときに呼ばれる
    public void visit(File file) {
        System.out.println(currentdir + "/" + file);
    }

    // ディレクトリを訪問したときに呼ばれる
    public void visit(Directory directory) {
        System.out.println(currentdir + "/" + directory);
        String savedir = currentdir;
        currentdir = currentdir + "/" + directory.getName();
        Iterator<Entry> it = directory.iterator();
        while (it.hasNext()) {
            Entry entry = it.next();
            entry.accept(this);
        }
        currentdir = savedir;
    }
}
```

entryへの訪問
＝ entryのaccept呼び出し
　自分自身を渡す
（相手に受け入れてもらうイメージ）

```java
public interface Element {
    public abstract void accept(Visitor v);
}
```

Visitorの受け入れ

インタフェースの実装
（Entry自体は抽象クラス
→ accepの実装はサブクラスに任せる

```java
public abstract class Entry implements Element {
    public abstract String getName();   // 名前を得る
    public abstract int getSize();      // サイズを得る

    // 文字列表現
    @Override
    public String toString() {
        return getName() + " (" + getSize() + ")";
    }
}
```

抽象クラスの継承（次ページ）

```java
public class File extends Entry
```

```java
public class Directory extends Entry
```

```java
public class Directory extends Entry {
    private String name; // ディレクトリの名前
    private List<Entry> dir = new ArrayList<Entry>();
    public Directory(String name) {
        this.name = name;
    }

    //抽象メソッドの実装
    @Override
    public String getName() {
        return name;
    }
    public int getSize() {
        int size = 0;
        Iterator<Entry> it = dir.iterator();
        while (it.hasNext()) {
            Entry entry = (Entry)it.next();
            size += entry.getSize();
        }
        return size;
    }
    public void accept(Visitor v) {
        v.visit(this);
    }


    public Entry add(Entry entry) {
        dir.add(entry);
        return this;
    }
    public Iterator<Entry> iterator() {
        return dir.iterator();
    }

}
```

```java
public class File extends Entry {
    private String name;
    private int size;
    public File(String name, int size) {
        this.name = name;
        this.size = size;
    }

    //抽象メソッドの実装
    public String getName() {
        return name;
    }
    public int getSize() {
        return size;
    }
    public void accept(Visitor v) {
        v.visit(this);
    }
}
```

訪問者の受け入れ

ディレクトリ階層の構築と訪問

lec05_visitor/*.java

コンパイル方法: javac lec05_visitor/Main.java
実行方法: java lec05_visitor/Main

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Making root entries...");
        Directory rootdir = new Directory("root");
        Directory bindir = new Directory("bin");
        Directory tmpdir = new Directory("tmp");
        Directory usrdir = new Directory("usr");
        rootdir.add(bindir);
        rootdir.add(tmpdir);
        rootdir.add(usrdir);
        bindir.add(new File("vi", 10000));
        bindir.add(new File("latex", 20000));
        rootdir.accept(new ListVisitor());

        System.out.println("");
        System.out.println("Making user entries...");
        Directory yuki = new Directory("yuki");
        Directory hanako = new Directory("hanako");
        Directory tomura = new Directory("tomura");
        usrdir.add(yuki);
        usrdir.add(hanako);
        usrdir.add(tomura);
        yuki.add(new File("diary.html", 100));
        yuki.add(new File("Composite.java", 200));
        hanako.add(new File("memo.tex", 300));
        tomura.add(new File("game.doc", 400));
        tomura.add(new File("junk.mail", 500));
        rootdir.accept(new ListVisitor());
    }
}
```

訪問1

User Entryの追加

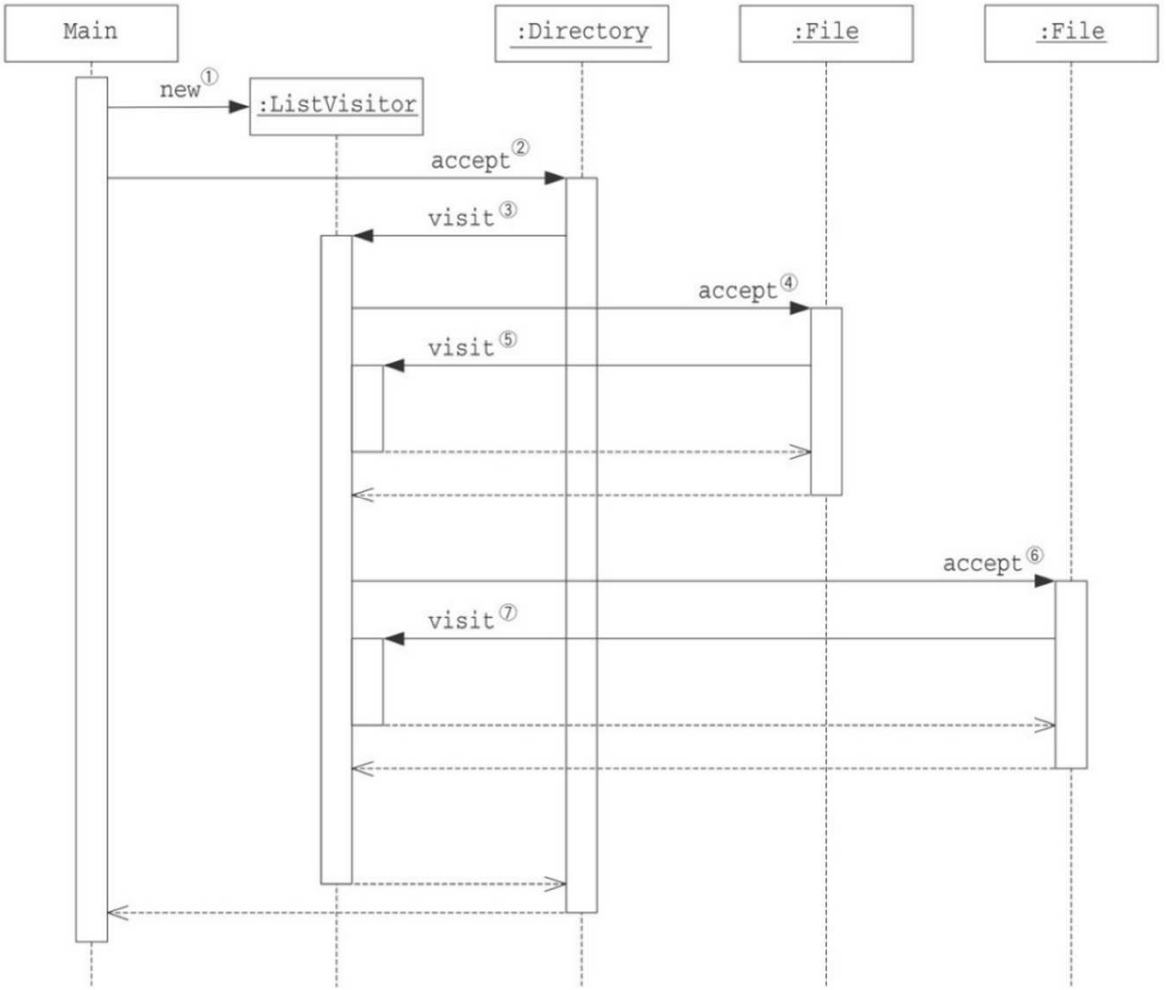
訪問2

```
Making root entries...
/root (30000)
/root/bin (30000)
/root/bin/vi (10000)
/root/bin/latex (20000)
/root/tmp (0)
/root/usr (0)

Making user entries...
/root (31500)
/root/bin (30000)
/root/bin/vi (10000)
/root/bin/latex (20000)
/root/tmp (0)
/root/usr (1500)
/root/usr/yuki (300)
/root/usr/yuki/diary.html (100)
/root/usr/yuki/Composite.java (200)
/root/usr/hanako (300)
/root/usr/hanako/memo.tex (300)
/root/usr/tomura (900)
/root/usr/tomura/game.doc (400)
/root/usr/tomura/junk.mail (500)
```

動作概要

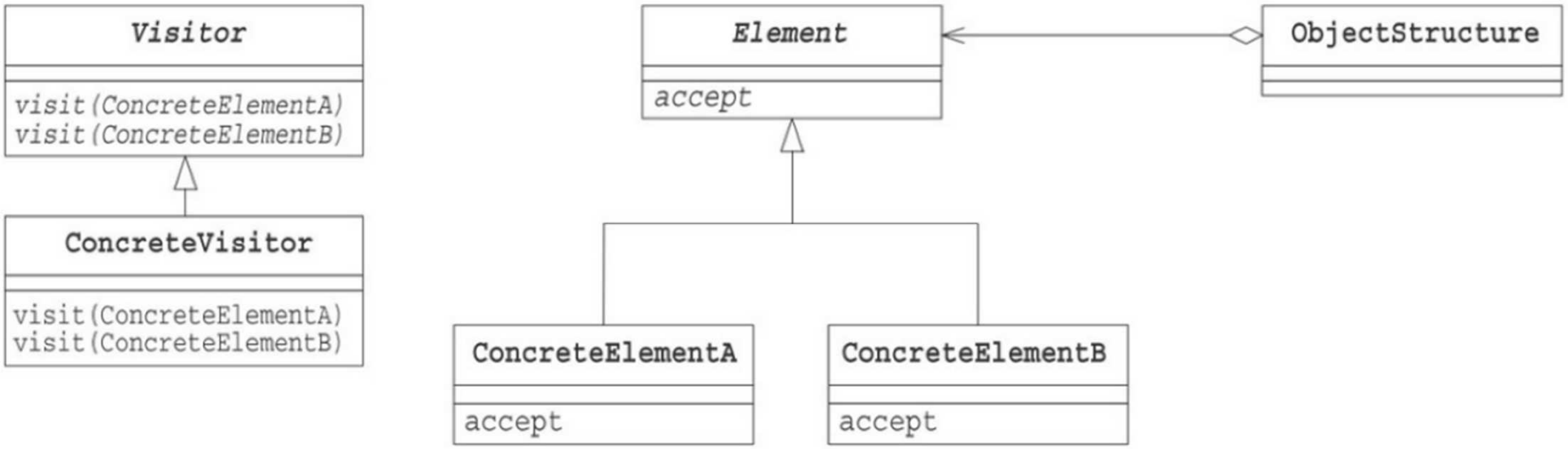
- Visitor側とElement側の相互呼び出し
- ポイント
  - Directory・Fileのインスタンスに対しては，acceptメソッドが呼び出される
  - acceptメソッドは，インスタンスごとに一回しか呼ばれない
  - 一つのVisitorインスタンスが visit(Directory)，visit(File)を処理する



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

Visitor パターンの登場人物

- Visitor（訪問者）の役：ConcreteElement役毎に「xxxを訪問した」というvisit(xxx)メソッドを宣言する．実装は，ConcreteVisitor役に任せる
- ConcreteVisitor（具体的訪問者）の役：Visitor役のAPIを実装する
- Element（要素）の役：Visitor役の訪問先を表す．訪問者を受け入れるacceptメソッドを宣言する．実装はConcreteElement役に任せる
- ConcreteElement（具体的要素）の役：Element役のAPIを実装する．
- ObjectStructure（オブジェクトの構造）の役：Element役の集合を扱う役
  - ConcreteVisitor役が個々のElement役を扱えるようなメソッドを備えている
  - サンプルプログラムでは，Directoryクラスがこれに相当
    - Directoryクラスは，ConcreteElement と ObjectStructureの両方の役を務めた



結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考えを広げるためのヒント

- ダブルディスパッチ（double dispatch：二重の振り分け）
  - ConcreteElement役とConcreteVisitor役の組によって実際の処理が決定
  - これを一般に ダブルディスパッチと呼ぶ
  - ConcreteElementでは，element.accept(visitor)
  - ConcreteVisitorでは，visitor.visit(element)
  - ※相互に，相手を引数にしたメソッドを呼び出している

- Visitorパターンの目的：「データ構造」と「処理」の分離
  - FileクラスやDirectoryクラスの部品としての独立性を高める
    - 部品の中に「処理」がある場合→処理の追加 = クラスの修正

- The Open-Closed Principle
  - 拡張（extension）については開かれている（open）が，修正（modification）については閉じられている（closed）べきとの主張
  - 「既存のクラスを修正せずに拡張できるようにせよ」
  - 詳細は自分で調べてみよう！
    - 例えば，https://stg-tud.github.io/sedc/Lecture/ss15/3.3-OCP.pdf
  - Visotorパターンの場合
    - ConcreteVisotor役の追加は簡単／ConcreteElement役の追加は困難

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

演習1

- lec05_visitor/*.java
- 以下の仕様を満たすCounfFileVisitorクラスを作成せよ
  - 指定されたディレクトリを階層的に訪問し，指定ディレクトリ以下に含まれるファイル（Fileクラスのインスタンス）の総数をカウントする.
  - 訪問後，getFileCounts()メソッドで，数えた数を返す

- 以下の仕様を満たすFileFindVisitorクラスを作成せよ
  - コンストラクタで，収集するファイルの拡張子を指定する
  - 指定されたディレクトリを階層的に訪問し，指定された拡張子のファイル（Fileクラスのインスタンス）を集める
  - 訪問後，getFoundFiles()メソッドで，集めたファイルに対するイテレータインスタンスを返す

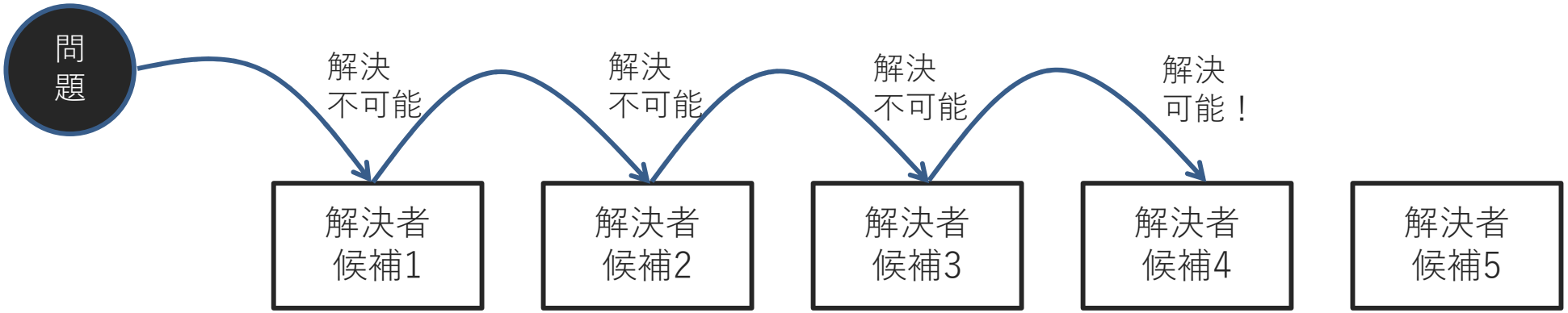
  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec05_visitor;

- （余力のある学生さん向け）Visitorパターンを適用したプログラム例を作成せよ

Chain of Responsibility パターン
～責任のたらい回し～

Chain of Responsibilityパターン

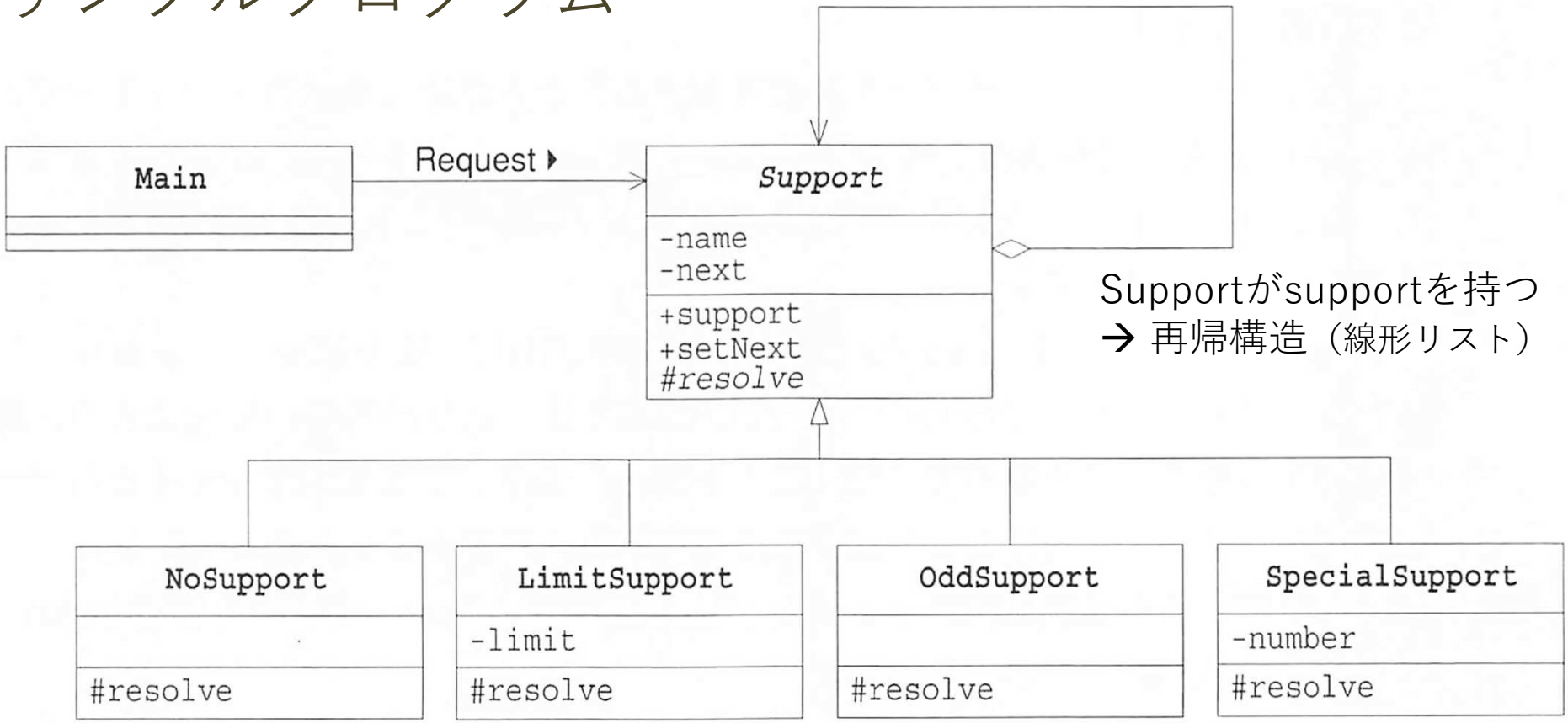
- Chain of responsibilityパターン：複数のオブジェクトを鎖のようにつなぎ，そのオブジェクトの鎖を順次渡り歩いて，目的のオブジェクトを決定する
- 責任の連鎖（たらい回し）



- 「要求を出す人」と「要求を処理する人」を緩やかにつなぐための考え方
  - このパターンを使わないと，「要求を出す人」が中央集権的に，各要求は誰が処理すべきかを管理しないといけない
- 利用シーン
  - 要求と処理の関係が流動的（今後，変わるかもしれない）場合
    → 新たな処理が発生したら，連鎖中の適切な位置に挿入するだけで良い
- 注意：このパターンは，「たらい回し」方式のため処理速度が遅い可能性がある．要求・処理の関係が固定的であるなら，単にifブロックで処理を決定した方が処理は速い．
- たらい回し先で循環が起きないように気を付けないと，無限ループに

サンプルプログラム

lec05_chain_of_responsibility/*.java



Supportがsupportを持つ
→ 再帰構造（線形リスト）

名　前	解　説
Trouble	発生したトラブルを表すクラス。トラブル番号（number）を持つ
Support	トラブルを解決する抽象クラス
NoSupport	トラブルを解決する具象クラス（常に「処理しない」）
LimitSupport	トラブルを解決する具象クラス（指定した番号未満のトラブルを解決）
OddSupport	トラブルを解決する具象クラス（奇数番号のトラブルを解決）
SpecialSupport	トラブルを解決する具象クラス（特定番号のトラブルを解決）
Main	Supportたちの連鎖を作り、トラブルを起こす動作テスト用のクラス

コンパイル方法: javac lec05_chain_of_responsibility/Main.java
実行方法:　　java lec05_chain_of_responsibility/Main

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

lec05_chain_of_responsibility/*.java

Trouble：トラブル番号を持つ

```java
public class Trouble {
    private int number;             // トラブル番号
    public Trouble(int number) {    // トラブルの生成
        this.number = number;
    }
    public int getNumber() {        // トラブル番号を得る
        return number;
    }
    public String toString() {      // トラブルの文字列表現
        return "[Trouble " + number + "]";
    }
}
```

Support：トラブルを解決する連鎖を作るための抽象クラス

```java
public abstract class Support {
    private String name;                    // このトラブル解決者の名前
    private Support next;                   // たらい回しの先
    public Support(String name) {           // トラブル解決者の生成
        this.name = name;
    }
    public Support setNext(Support next) {  // たらい回しの先を設定
        this.next = next;
        return next;
    }
    public void support(Trouble trouble) {  // トラブル解決の手順
        if (resolve(trouble)) {
            done(trouble);
        } else if (next != null) {
            next.support(trouble);
        } else {
            fail(trouble);
        }
    }
    public String toString() {              // 文字列表現
        return "[" + name + "]";
    }
    protected abstract boolean resolve(Trouble trouble); // 解決用メソッド
    protected void done(Trouble trouble) {  // 解決
        System.out.println(trouble + " is resolved by " + this + ".");
    }
    protected void fail(Trouble trouble) {  // 未解決
        System.out.println(trouble + " cannot be resolved.");
    }
}
```

線形リストを繋ぐイメージ

(1)解決を試みる
(2)解決できなければnextに任せる
(3)Nextがいなければfail

継承して実装

```java
public class NoSupport extends Support {
    public NoSupport(String name) {
        super(name);
    }
    protected boolean resolve(Trouble trouble) {
        return false; // 自分は何も処理しない
    }
}
```

```java
public class LimitSupport extends Support {
    private int limit;
    public LimitSupport(String name, int limit) {
        super(name);
        this.limit = limit;
    }
    protected boolean resolve(Trouble trouble) {
        if (trouble.getNumber() < limit) {
            return true;
        } else {
            return false;
        }
    }
}
```

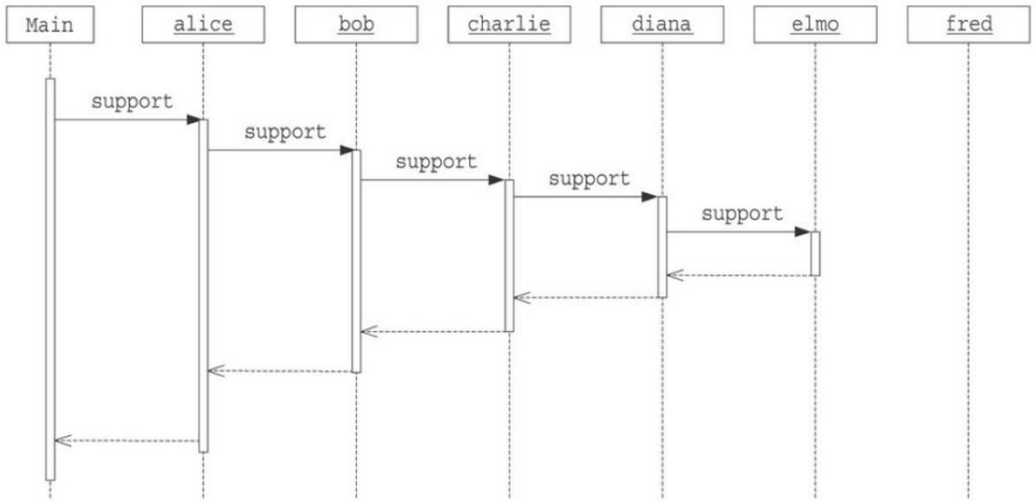
```java
public class OddSupport extends Support {
    public OddSupport(String name) {
        super(name);
    }
    protected boolean resolve(Trouble trouble) {
        if (trouble.getNumber() % 2 == 1) {
            return true;
        } else {
            return false;
        }
    }
}
```

```java
public class SpecialSupport extends Support {
    private int number;
    public SpecialSupport(String name, int number) {
        super(name);
        this.number = number;
    }
    protected boolean resolve(Trouble trouble) {
        if (trouble.getNumber() == number) {
            return true;
        } else {
            return false;
        }
    }
}
```

lec05_chain_of_responsibility/*.java

コンパイル方法: javac lec05_chain_of_responsibility/Main.java
実行方法: java lec05_chain_of_responsibility/Main

```java
public class Main {
    public static void main(String[] args) {
        Support alice   = new NoSupport("Alice");
        Support bob     = new LimitSupport("Bob", 100);
        Support charlie = new SpecialSupport("Charlie", 429);
        Support diana   = new LimitSupport("Ðiana", 200);
        Support elmo    = new OddSupport("Elmo");
        Support fred    = new LimitSupport("Fred", 300);
        // 連鎖の形成
        alice.setNext(bob).setNext(charlie).setNext(diana).setNext(elmo).setNext(fred);
        // さまざまなトラブル発生
        for (int i = 0; i < 500; i += 33) {
            alice.support(new Trouble(i));
        }
    }
}
```

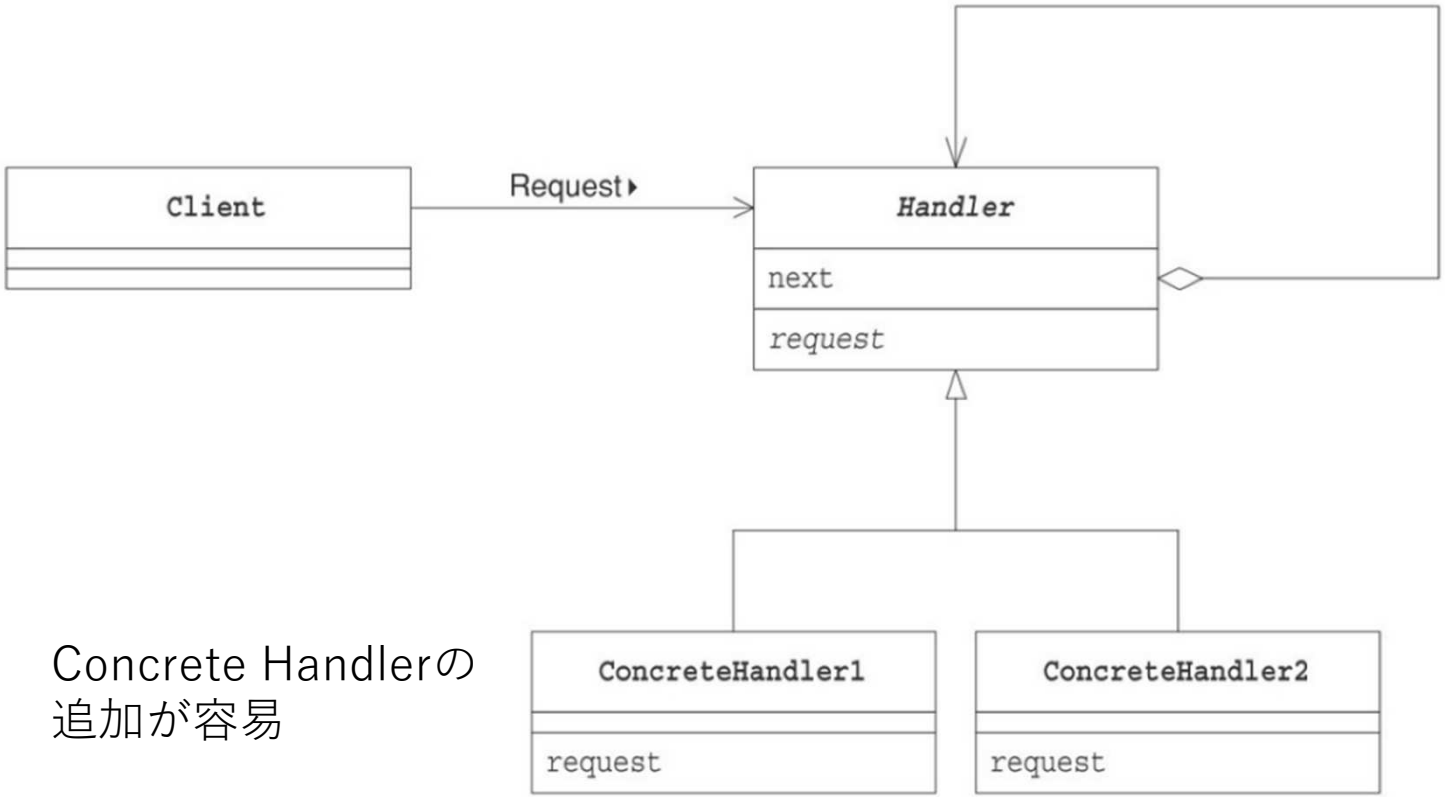


結城浩 『Java言語で学ぶデザインパターン入門第３版』
SBクリエイティブ(2021)より

```
[Trouble 0] is resolved by [Bob].
[Trouble 33] is resolved by [Bob].
[Trouble 66] is resolved by [Bob].
[Trouble 99] is resolved by [Bob].
[Trouble 132] is resolved by [Diana].
[Trouble 165] is resolved by [Diana].
[Trouble 198] is resolved by [Diana].
[Trouble 231] is resolved by [Elmo].
[Trouble 264] is resolved by [Fred].
[Trouble 297] is resolved by [Elmo].
[Trouble 330] cannot be resolved.
[Trouble 363] is resolved by [Elmo].
[Trouble 396] cannot be resolved.
[Trouble 429] is resolved by [Charlie].
[Trouble 462] cannot be resolved.
[Trouble 495] is resolved by [Elmo].
```

Chain of Responsibilityの登場人物

- Handler（処理者）の役
  - 要求を処理するインタフェース（API）を定める役
  - 「次の人」を保持し，自分が処理できない要求が来たら，その人に回す
- ConcreteHandler（具体的処理者）の役
  - 要求を処理する具体的な役
- Client（要求者）の役
  - ConcreteHandler役に要求を出す役（サンプルプログラムではMain）



nextとしてHandlerを持つ
→ 再帰構造

Concrete Handlerの
追加が容易

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

考えを広げるためのヒント

- 要求を出す人と要求を処理する人を緩やかに結びつける
  - 要求を出す人（Client役） と 要求を処理する人（ConcreteHandler役）を緩やかに結びつける
  - Client役は，要求を処理する「最初に人」にお願いすればよいだけ
    - どの人がどんな要求を処理するかを把握する必要はない
    - 部品としての独立性が高まる

- 動的に連鎖の形態を変える
  - サンプロプログラムでは，最初に要求を処理する人を繋いだ→静的
  - 状況に応じて，連鎖の要素を選択することも可能→動的
    - （複雑になるが）要求を受け取った人が，処理を依頼する相手を動的に選択することも可能

- 自分の仕事に集中できる
  - 要求を処理する人は，自分ができる仕事だけをする．
  - 個々のプログラムはシンプルになる

演習2

- lec05_chain_of_responsibility/*.java
  - 以下の内容のConcreteHandler役を作成せよ．またMainを改変し，動作確認を行いなさい．
  - Geoffrey： 11以上15以下の番号のトラブルを解決できる
  - Henry：事前に指定された（複数の）番号のトラブルのみを解決できる
    - 番号を指定は，インスタンス生成時にリストor配列を渡すことで行う
  - Iris：やる気があるときはすべての番号を解決できる（やる気のないときは，いかなる番号も解決できない）
    - Irisは，「解決の依頼がある都度，15%の確率でやる気がでる」とする
  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec05_chain_of_responsibility;
- （余力のある学生さん向け）Chain of Responsibilityパターンを適用したプログラム例を作成せよ

State パターン
～状態をクラスとして表現する～

State パターン

- オブジェクト指向プログラミング
  - プログラムする対象を「クラス」として表現する.
  - どの様なものをクラスとして表現するか？
    - 人・動物などの具体的なもの，概念・状態など抽象的なもの

- Stateパターン：「状態」をクラスとして表現する
  - State = 状態 = ものごとのありさま
  - 複雑な条件分岐が排除できる

- 「状態」をクラスにする目的
  - 「状態の変化」＝クラスの切り替え
  - 「状態の追加」＝（サブ）クラスの追加

- （Processingで）ゲームを作る場合で考えると．．
  - 状態：初期画面，ゲームステージ１，ゲームステージ２，エンディング画面
    - 各画面でのクリック時の動作 → 全部mouseClickedメソッドに書く？
      - 「状態」の判定が必要：状態数が増えると管理も大変
      - 新しい状態の追加：mouseClickedメソッドを書き換える
    - Stateパターンでスッキリとしたプログラムを！

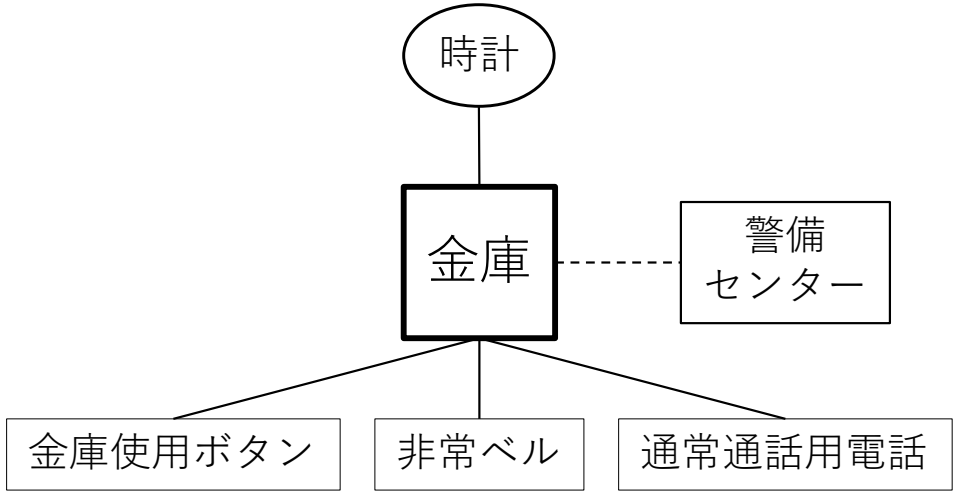
結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

使用シーン

lec05_state/*.java

コンパイル方法: javac lec05_state/Main.java
実行方法: java lec05_state/Main

- 金庫警備システム
  - 時間ごとに警備の状態が変化する金庫警備システム



- 金庫が一つある
- 金庫は警備センターと接続されている
- 金庫には非常ベル・通常電話が接続されている
- 金庫には時計があり，時間を監視している

- 昼間：09:00 – 16:59
  - 金庫は昼間だけ利用できる
  - 金庫を利用すると，警備センターに使用記録が残る
  - 電話を利用すると，警備センターが呼び出される
  - 非常ベルを利用すると，警備センターに非常ベルの通報が行く

- 夜間：17:00－23:59，0:00－08:59 (昼以外の時間)
  - 金庫を利用すると，警備センターに非常事態の通報が行く
  - 電話を利用すると，警備センターの留守録が呼び出される
  - 非常ベルを利用すると，警備センターに非常ベルの通報が行く

金庫使用ボタンや
通常通話用電話の動作が，
昼と夜で異なる

Processing でゲームの場合．．
クリック・キー操作の動作が
ステージで異なる

- 実行してみよう：Windowが立ち上がるので，ボタンを押してみる

State パターンを使う・使わない疑似コード

使わない場合．．

```
class 警備システム {

  void 金庫使用時のメソッド( ) {
    if( 昼間 ) {
    警備センターに利用の記録

    } else if( 夜間 ){
    警備センターに非常事態の通報
    }
  }

  void 非常ベル使用時のメソッド( ) {
    警備センターに非常ベルの通報

  }

  void 電話使用時のメソッド( ){
    if( 昼間 ){
    警備センターの呼び出し

    } else if( 夜間 ){
    警備センターの留守録の呼び出し

    }
  }
}
```

昼夜がif文中に登場
各メソッド内で状態をチェック

使う場合．．

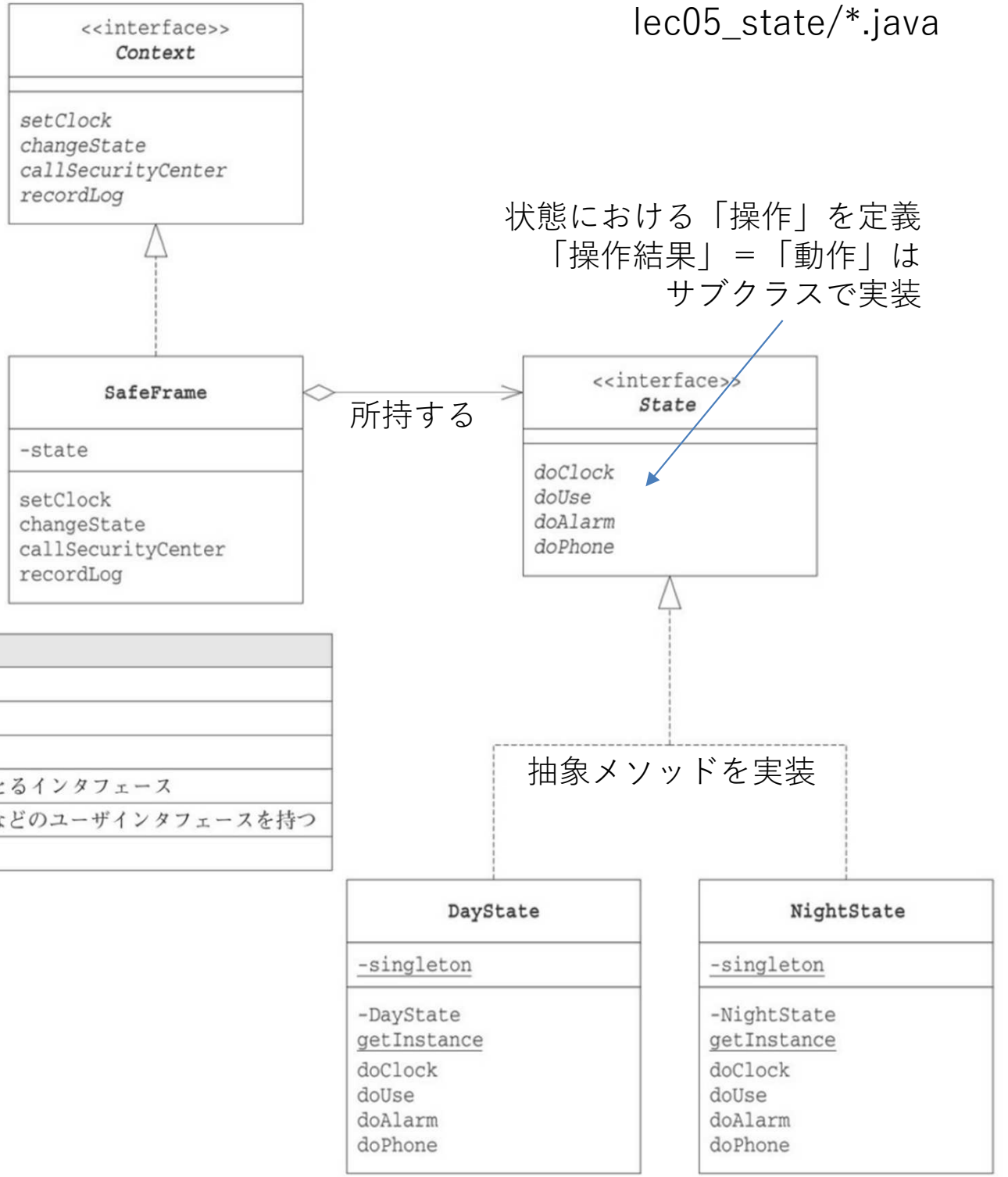
```
class 昼間 implements 状態 {
  void 金庫使用時のメソッド( ) {
    警備センターに利用の記録
  }
  void 非常ベル使用時のメソッド( ) {
    警備センターに非常ベルの通報
  }
  void 電話使用時のメソッド( ){
    警備センターの呼び出し
  }
}

class 夜間 implments 状態 {
  void 金庫使用時のメソッド( ) {
    警備センターに非常事態の通報
  }
  void 非常ベル使用時のメソッド( ) {
    警備センターに非常ベルの通報
  }
  void 電話使用時のメソッド( ){
    警備センターの留守録の呼び出し
  }
}
```

昼，夜は（状態を実装した）クラス
状態チェックのif文は存在しない

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

lec05_state/*.java



名　前	解　説
State	金庫の状態を表すインタフェース
DayState	Stateを実装しているクラス。昼間の状態を表す
NightState	Stateを実装しているクラス。夜間の状態を表す
Context	金庫の状態変化を管理し、警備センターとの連絡をとるインタフェース
SafeFrame	Contextを実装しているクラス。ボタンや画面表示などのユーザインタフェースを持つ
Main	動作テスト用のクラス

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

各出来事（アクション）に対応して呼び出されるインタフェースを規定する
Contextは，状態管理を行っているインタフェース（後述）

```java
public interface State {
    public abstract void doClock(Context context, int hour);     // 時刻設定
    public abstract void doUse(Context context);                 // 金庫使用
    public abstract void doAlarm(Context context);               // 非常ベル
    public abstract void doPhone(Context context);               // 通常通話
}
```

昼間の状態：Stateの各メソッドを実装

```java
public class DayState implements State {
    private static DayState singleton = new DayState();
    private DayState() {                                         // コンストラクタはprivate
    }
    public static State getInstance() {                          // 唯一のインスタンスを得る
        return singleton;
    }
    public void doClock(Context context, int hour) {             // 時刻設定
        if (hour < 9 || 17 <= hour) {
            context.changeState(NightState.getInstance());
        }
    }
    public void doUse(Context context) {                         // 金庫使用
        context.recordLog("金庫使用(昼間)");
    }
    public void doAlarm(Context context) {                       // 非常ベル
        context.callSecurityCenter("非常ベル(昼間)");
    }
    public void doPhone(Context context) {                       // 通常通話
        context.callSecurityCenter("通常の通話(昼間)");
    }
    public String toString() {                                   // 文字列表現
        return "[昼間]";
    }
}
```

この部分は，Singletonパターン
インスタンスを
丁度1つ作る

if文による状態チェックは不要

各メソッドは，context
（＝警備センターとやり取りする
インスタンス）のメソッドを
呼び出す

各出来事（アクション）に対応して呼び出されるインタフェースを規定する
Contextは，状態管理を行っているインタフェース（後述）

```java
public interface State {
    public abstract void doClock(Context context, int hour);     // 時刻設定
    public abstract void doUse(Context context);                 // 金庫使用
    public abstract void doAlarm(Context context);               // 非常ベル
    public abstract void doPhone(Context context);               // 通常通話
}
```

夜間の状態：Stateの各メソッドを実装

```java
public class NightState implements State {
    private static NightState singleton = new NightState();
    private NightState() {                                       // コンストラクタはprivate
    }
    public static State getInstance() {                          // 唯一のインスタンスを得る
        return singleton;
    }
    public void doClock(Context context, int hour) {             // 時刻設定
        if (9 <= hour && hour < 17) {
            context.changeState(DayState.getInstance());
        }
    }
    public void doUse(Context context) {                         // 金庫使用
        context.callSecurityCenter("非常：夜間の金庫使用！");
    }
    public void doAlarm(Context context) {                       // 非常ベル
        context.callSecurityCenter("非常ベル(夜間)");
    }
    public void doPhone(Context context) {                       // 通常通話
        context.recordLog("夜間の通話録音");
    }
    public String toString() {                                   // 文字列表現
        return "[夜間]";
    }
}
```

この部分は，
Singletonパターン
インスタンスを
丁度1つ作る

状態を管理し，警備センターとの連絡を取るインタフェース

```
public interface Context {

    public abstract void setClock(int hour);                    // 時刻の設定
    public abstract void changeState(State state);              // 状態変化
    public abstract void callSecurityCenter(String msg);        // 警備センター警備員呼び出し
    public abstract void recordLog(String msg);                 // 警備センター記録
}
```

```
public class SafeFrame extends Frame implements ActionListener, Context {
    private TextField textClock = new TextField(60);          // 現在時刻表示
    private TextArea textScreen = new TextArea(10, 60);       // 警備センター出力
    private Button buttonUse = new Button("金庫使用");         // 金庫使用ボタン
    private Button buttonAlarm = new Button("非常ベル");       // 非常ベルボタン
    private Button buttonPhone = new Button("通常通話");       // 通常通話ボタン
    private Button buttonExit = new Button("終了");            // 終了ボタン

    private State state = DayState.getInstance();             // 現在の状態
                        …

    // ボタンが押されたらここに来る
    public void actionPerformed(ActionEvent e) {
        System.out.println(e.toString());
        if (e.getSource() == buttonUse) {                     // 金庫使用ボタン
            state.doUse(this);
        } else if (e.getSource() == buttonAlarm) {            // 非常ベルボタン
            state.doAlarm(this);
        } else if (e.getSource() == buttonPhone) {            // 通常通話ボタン
            state.doPhone(this);
        } else if (e.getSource() == buttonExit) {             // 終了ボタン
            System.exit(0);
        } else {
            System.out.println("?");
        }
    }
```

stateのメソッドを呼び出す.
stateを入れ替えれば,
動作が変わる

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

SafeFrameの続き

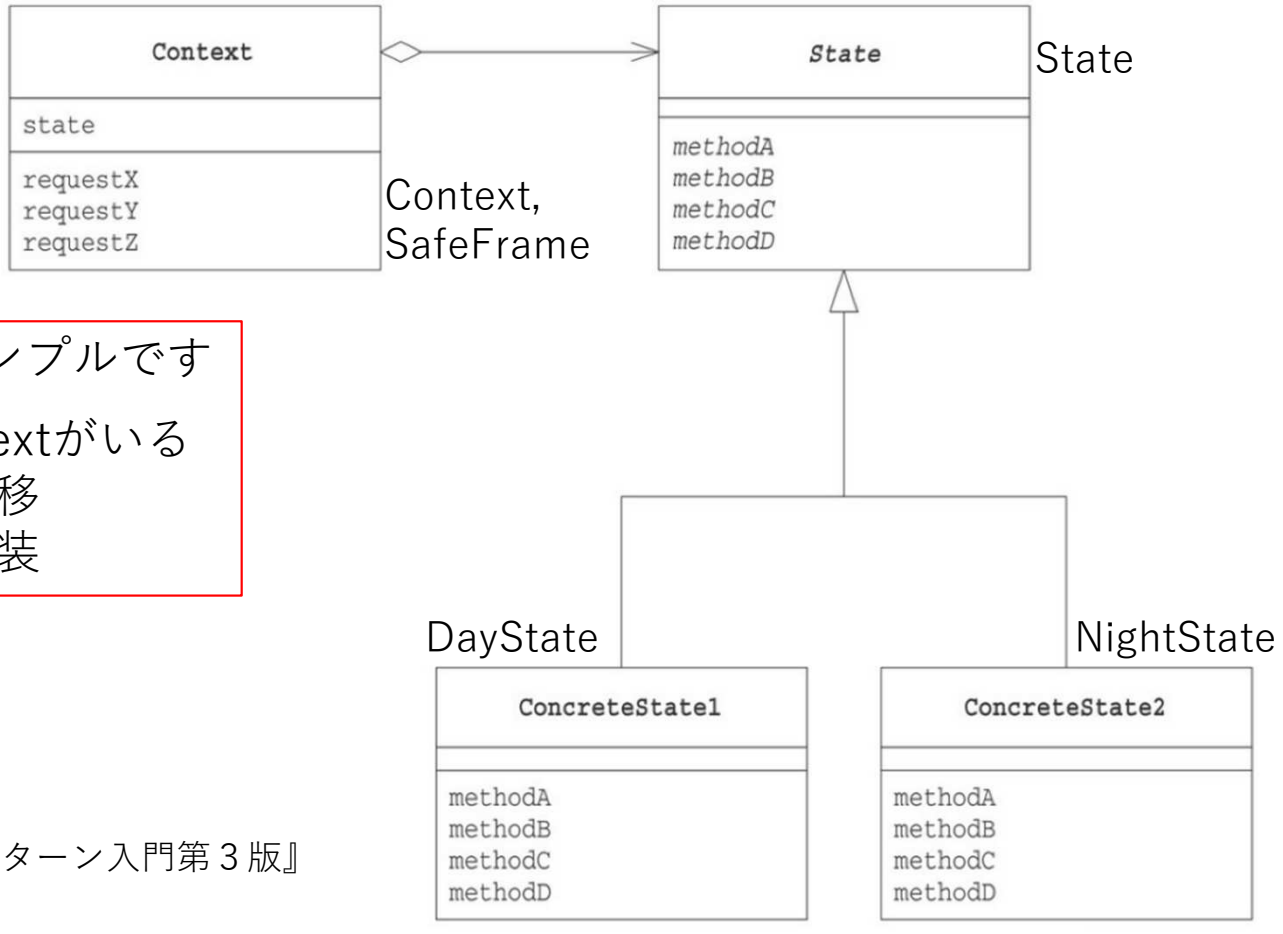
```
        ...
    // 時刻の設定
    public void setClock(int hour) {
        String clockstring = "現在時刻は";
        if (hour < 10) {
            clockstring += "0" + hour + ":00";
        } else {
            clockstring += hour + ":00";
        }
        System.out.println(clockstring);
        textClock.setText(clockstring);
        state.doClock(this, hour);
    }
    // 状態変化
    public void changeState(State state) {
        System.out.println(this.state + "から" + state + "へ状態が変化しました。");
        this.state = state;
    }
    // 警備センター警備員呼び出し
    public void callSecurityCenter(String msg) {
        textScreen.append("call! " + msg + "¥n");
    }
    // 警備センター記録
    public void recordLog(String msg) {
        textScreen.append("record ... " + msg + "¥n");
    }
```

stateを設定しなおす＝状態の遷移
stateを変えることで，実際の動作が変わる

stateから
呼び出される
メソッド

State パターンの登場人物

- State（状態）の役：Stateインタフェース
  - 状態に依存した振る舞いをするメソッドの集まり
- ConcreteState（具体的な状態）の役： DayStateクラス，NightStateクラス
  - 具体的な個々の状態を表現する役
  - State役で定められたインタフェースを実装
- Context（状況，文脈）の役：Contextインタフェース，SafeFrameクラス
  - Context役は，「現在の状態」を表すConcreteState役を持つ
  - Stateパターンの利用者に必要なインタフェースを与える



本質的な部分は意外とシンプルです

stateを持つ（使う）Contextがいる
stateの切り替え＝状態遷移
各state毎にメソッドを実装

結城浩 『Java言語で学ぶデザインパターン入門第３版』
SBクリエイティブ(2021)より

考えを広げるためのヒント

- 分割して統治せよ（divide and conqure）
  - 大きく難しい問題を一つ解く代わりに，小さく易しい問題をたくさん解く
    - Stateパターン：状態をクラスで表現
      - 個々の具体的な状態を別々のクラスとして表現＝問題の分割
  - Stateパターンを使わない疑似コード
    - 各メソッド中で，if文による状態のチェックが必要
      - 状態数≒分岐数．新しい状態の追加＝既存プログラムの修正

- 状態遷移は誰が管理するべきか
  - 各ConcreteState役が管理する
    - 良い点：そのクラスだけを見れば，遷移先や条件が分かる
    - 悪い点：遷移先のクラスを知らなければならない
  - Context役が（集中的に）管理する
    - 良い点：ConcreteState役間の独立性が高まる
    - 悪い点：Context役がすべての状態遷移を把握しなければならない

- 新しい「状態」の追加 と 新しい「状態依存処理」の追加
  - 状態の追加は容易：状態遷移部（＝他のConcreteState役との接点）の実装には注意
  - 状態依存処理の追加は困難：既にあるConcreteState役すべての修正が必要

結城浩 『Java言語で学ぶデザインパターン入門第３版』 SBクリエイティブ(2021)より

演習3

- lec05_state/*.java
- サンプルプログラムに「昼食時」（12:00～12:59）という新たな状態を追加する
  - 昼食時，金庫を使用すると，警備センターに非常事態の通報が行く
  - 昼食時，非常ベルを使用すると，警備センターに非常ベルの通報が行く
  - 昼食時，電話を使用すると，警備センターの留守録が呼び出される．

  - ※指定ディレクトリ内で作成する
  - ※パッケージ宣言に注意
    - package lec05_state;

- （余力のある学生さん向け）Stateパターンを適用したプログラム例を作成せよ

- （余力のある学生さん向け）Processingを使って簡単なゲームを作ることを考える
  - 以下の3つのステージを考える．どのように実装すべきか設計してみよう
  - 初期ステージ（InitialStage）：背景色を単色で塗りつぶす
    - マウスクリック：ゲームステージへの移行
    - キーボード操作：塗り色を変更する
  - ゲームステージ(GameStage)：（10回跳ねる間に）動くボールをクリックするゲーム
    - マウスクリック：ボールクリックでエンディングステージへ移行／失敗時は初期ステージへ
    - キーボード操作：無効（何もしない）
  - エンディングステージ(EndingStage)：複数のボールが動く
    - マウスクリック：無効（何もしない）
    - キーボード操作：初期ステージへ戻る